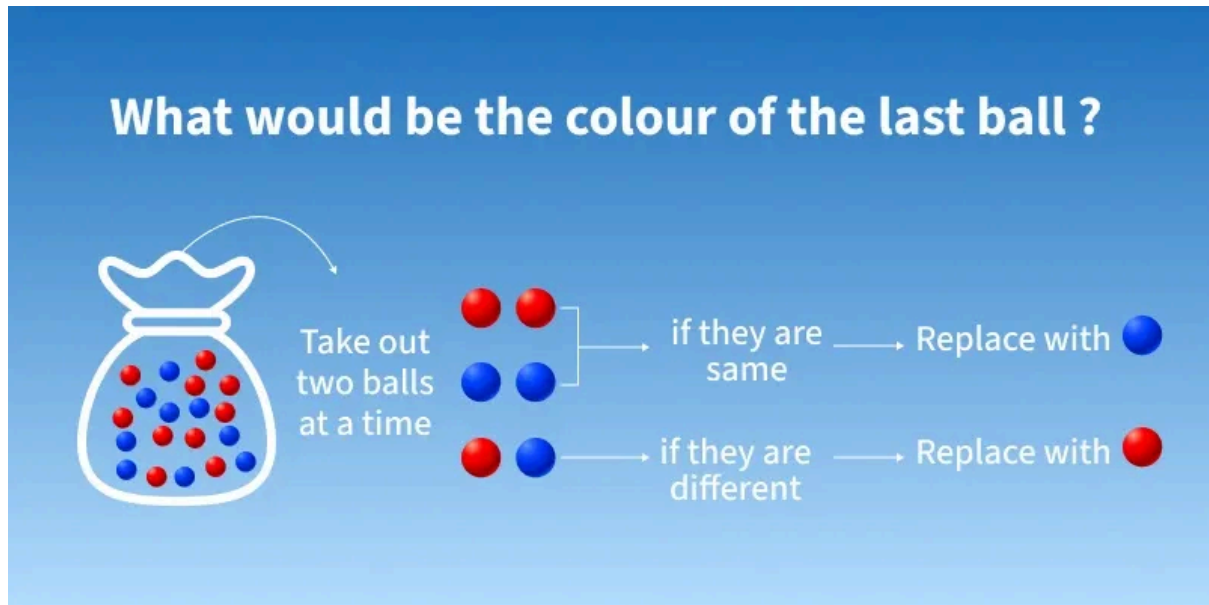


Question-1:

A bag contains 20 Red balls and 16 Blue balls. Remove any two balls from the bag. Each operation removes two balls and adds one ball back to the bag, and the process continues until only one ball remains.

What will be the color of the last remaining ball?



Solution:

Step 1: Check the number of Red balls

Initially:

- 20 Red balls
- 16 Blue balls

20 is an even number.

Step 2: What happens to Red balls?

When two balls are removed:

- Red + Red $\rightarrow$ Red decreases by 2
- Blue + Blue $\rightarrow$ Red unchanged

- Red + Blue $\rightarrow$ Red unchanged

So the number of Red balls either decreases by 2 or stays the same.

This means the number of Red balls always remains even.

Step 3: The last ball

At the end, only one ball remains.

If it were Red, then the number of Red balls would be 1 (odd).

But we have already shown that the number of Red balls always remains even.

Since this is not possible, the last ball cannot be Red.

So, the only possibility is that the last remaining ball is **Blue**.

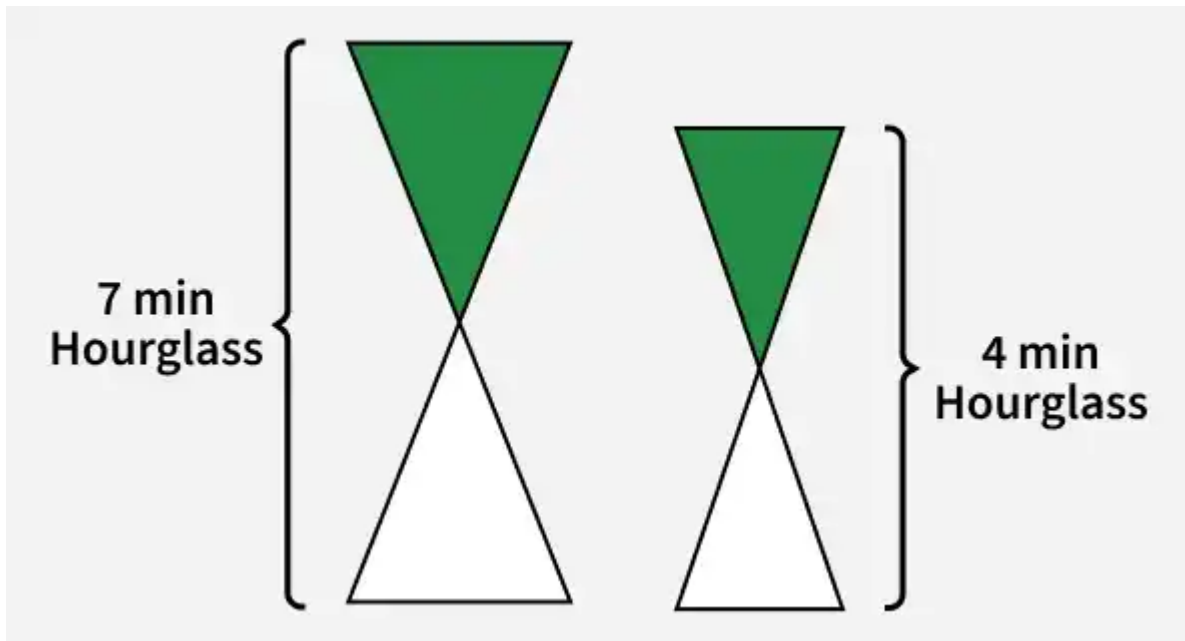
Core Insight

For this set of replacement rules:

- *If the initial number of Red balls is odd, the last ball is Red.*
- *If it is even, the last ball is Blue.*

Question-2:

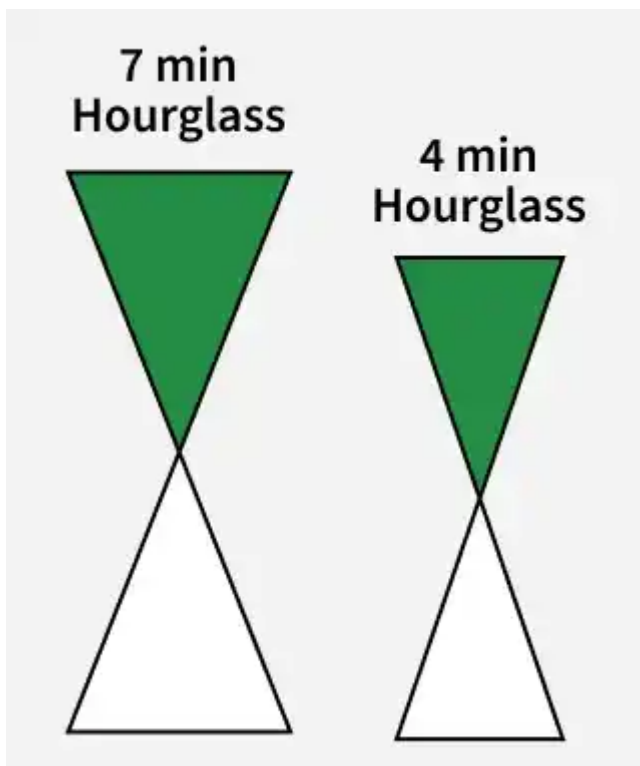
Given two hourglasses of 4 minutes and 7 minutes, the task is to measure 9 minutes.



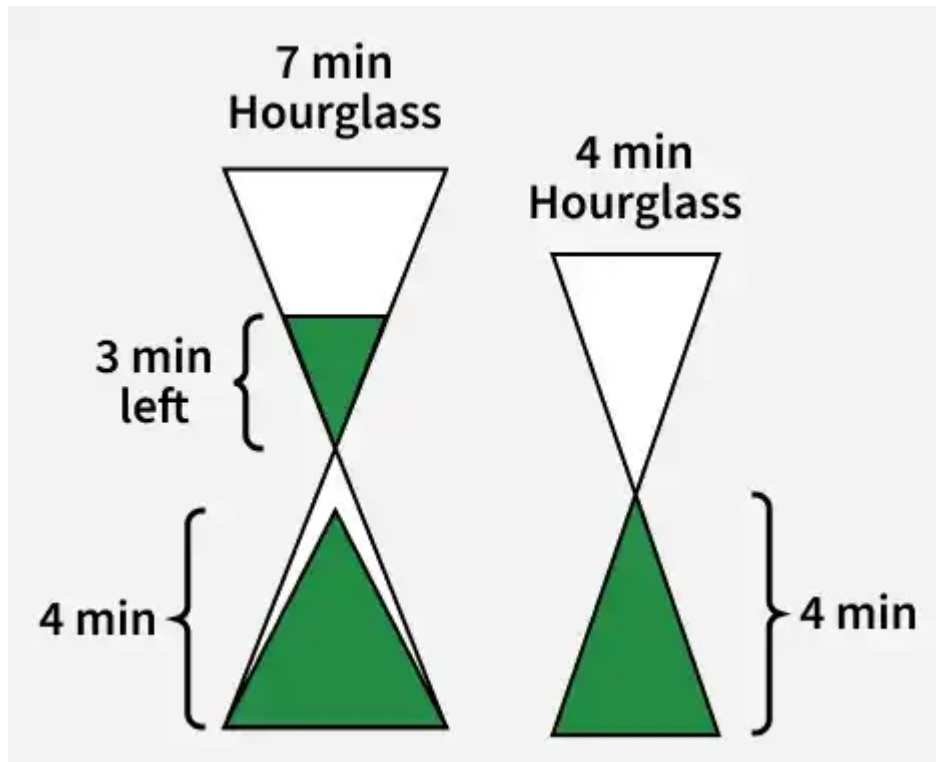
Solution:

Solution:

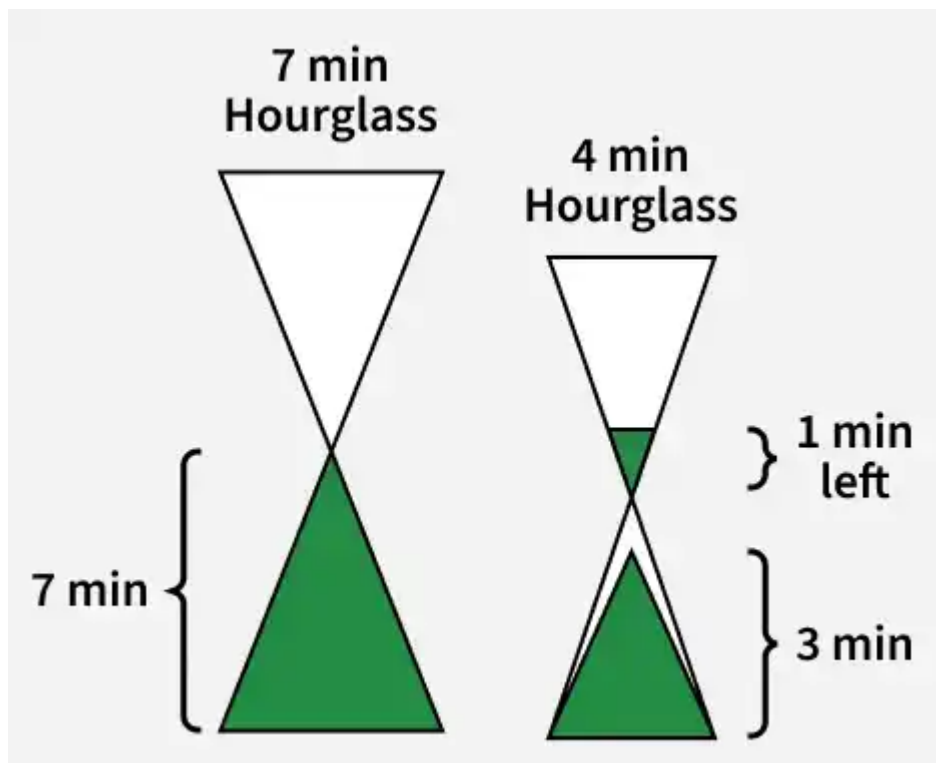
1. At 0 minutes: Start both hourglasses at the same time.



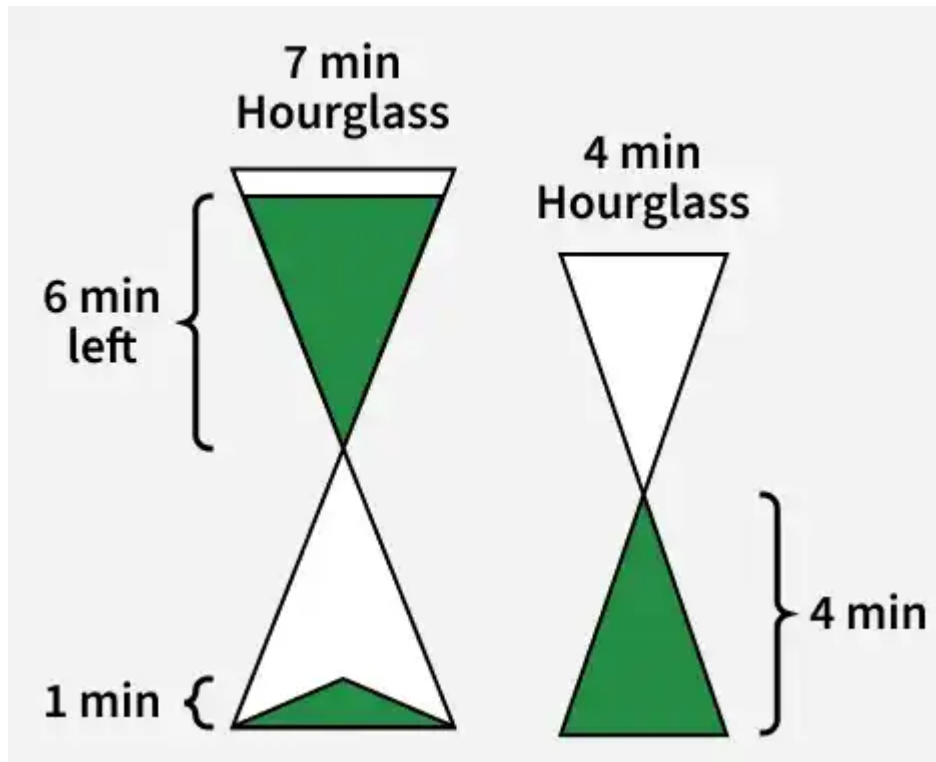
2. At 4 minutes: 4 minutes hourglass runs out and flip it. 7 minutes hourglass is left with 3 minutes.



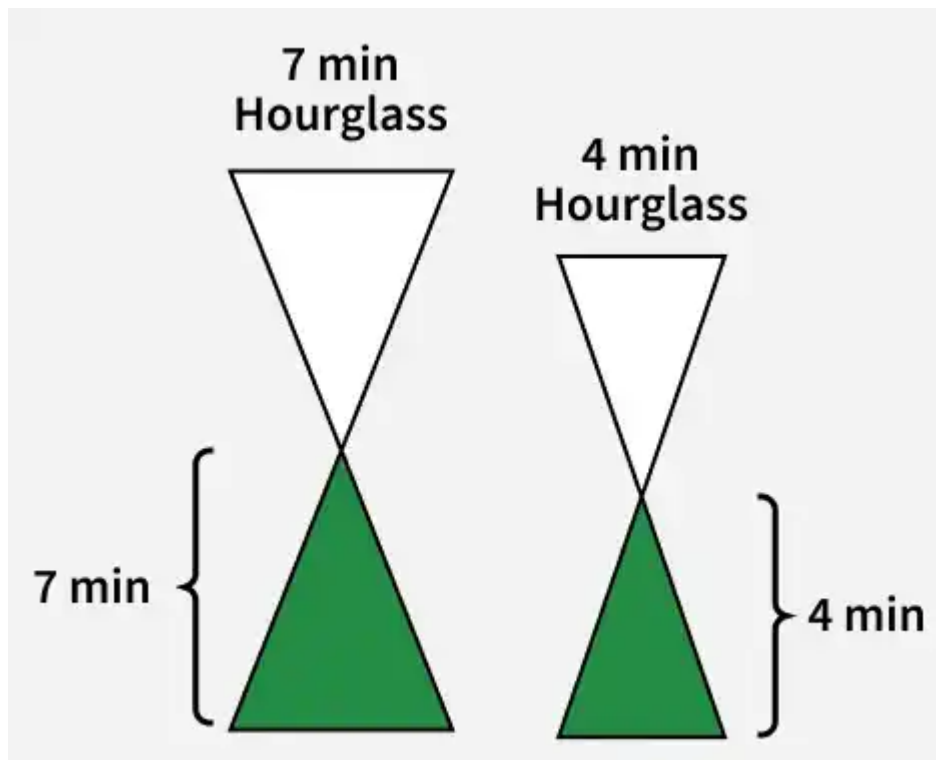
3. At 7 minutes: 4 minutes hourglass is left with 1 minute. 7 minutes hourglass runs out and flip it.



4. At 8 minutes: 4 minutes hourglass runs out and 7 is filled with 6 minutes and 1 minute on the other side. Flip it as the sand is left with 1 minute.



5. At 9 minutes: 7 minutes hourglass becomes empty from above side.



Hence we measured 9 minutes.

Question-3:

You are given an array of N integers. You need to answer multiple queries.

For each query $[L, R]$, find the **minimum/maximum/sum/product** in the array from index L to index R (both inclusive).

Since there can be many queries, you must **construct a Segment Tree** to answer each query efficiently.

Input

N
 $A[0] A[1] A[2] \dots A[N-1]$
 Q
 $L_1 R_1$
 $L_2 R_2$
 $\dots$
 $L_Q R_Q$

Example

Input

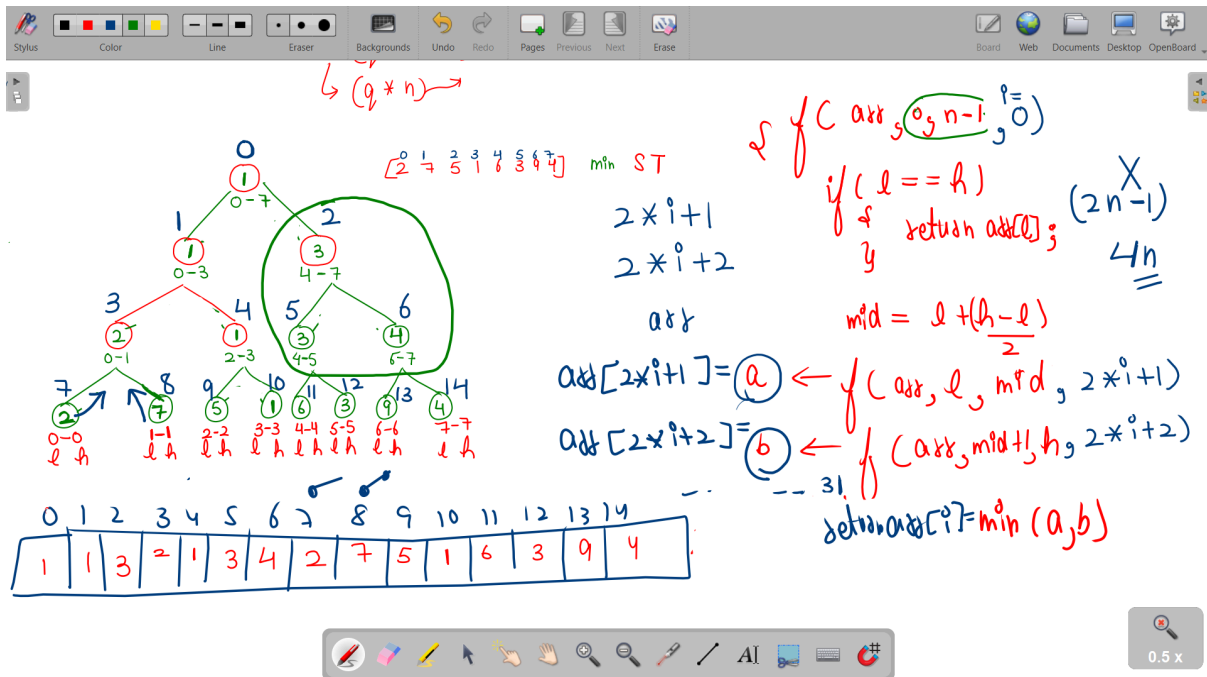
6
5 2 8 1 6 3
3
0 2
1 4
3 5

Output

2
1
1

Task

Construct a **Minimum Segment Tree** for the given array and use it to answer all range minimum queries.



Solution:

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
void build(vector<int>& A, vector<int>& tree, int low, int high, int index) {
```

```
    // Base case: leaf node
```

```
    if (low == high) {
```

```
        tree[index] = A[low];
```

```
        return;
```

```
    }
```

```
    int mid = (low + high) / 2;
```

```
    // Build left subtree
```

```
    build(A, tree, low, mid, 2 * index + 1);
```

```
    // Build right subtree
```

```
    build(A, tree, mid + 1, high, 2 * index + 2);
```

```
    // Merge: minimum of both children
```

```
    tree[index] = min(tree[2 * index + 1],
```

```
                      tree[2 * index + 2]);
```

```
}
```

```
int query(vector<int>& tree, int low, int high,
```

```
         int L, int R, int index) {
```

```
    // Completely outside
```

```

    if (high < L || low > R) {
        return INT_MAX;
    }

    // Completely inside
    if (L <= low && high <= R) {
        return tree[index];
    }

    // Partially overlapping
    int mid = (low + high) / 2;

    int leftAnswer = query(tree, low, mid, L, R, 2 * index + 1);
    int rightAnswer = query(tree, mid + 1, high, L, R, 2 * index + 2);

    return min(leftAnswer, rightAnswer);
}

int main() {
    int N;
    cin >> N;

    vector<int> A(N);

    for (int i = 0; i < N; i++) {
        cin >> A[i];
    }

    // Segment tree size
    vector<int> tree(4 * N);

    // Build segment tree
    build(A, tree, 0, N - 1, 0);

    int Q;
    cin >> Q;

    while (Q--) {
        int L, R;
        cin >> L >> R;

        cout << query(tree, 0, N - 1, L, R, 0) << endl;
    }

    return 0;
}

```

Question-4:

An online shopping platform has N products arranged in a fixed order. Each product has a discount factor represented by $A[i]$.

For a promotional campaign covering products from index L to R , the combined discount factor is calculated as the product of all discount factors in that range.

For every query $[L, R]$, calculate:

$$A[L] \times A[L+1] \times \dots \times A[R]$$

Since there can be a large number of queries, you must construct a Product Segment Tree to answer each query efficiently.

Input

N

$A[0] A[1] A[2] \dots A[N-1]$

Q

$L_1 R_1$

$L_2 R_2$

...

$L_Q R_Q$

Output

For each query, print the product of all elements from index L to R , both inclusive.

Example:

15

2 3 5 4 6 2 7 3 5 2 4 6 3 2 5

Constraints

$$1 \leq N, Q \leq 10^5$$

$$1 \leq A[i] \leq 10^4$$

$$0 \leq L \leq R < N$$

Solution:

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
void build(vector<long long>& A,  
           vector<long long>& tree,  
           int low, int high, int index) {
```

```
    // Base case
```

```
    if (low == high) {  
        tree[index] = A[low];  
        return;  
    }
```

```
    int mid = (low + high) / 2;
```

```
    // Build left subtree
```

```
    build(A, tree, low, mid, 2 * index + 1);
```

```
    // Build right subtree
```

```
    build(A, tree, mid + 1, high, 2 * index + 2);
```

```
    // Merge
```

```
    tree[index] = tree[2 * index + 1] *  
        tree[2 * index + 2];
```

```
}
```

```
long long query(vector<long long>& tree,
```

```
    int low, int high,
```

```
    int L, int R,
```

```
    int index) {
```

```
    // Completely non-overlapping
```

```
    if (high < L || low > R) {
```

```
        return 1;
```

```
    }
```

```
    // Completely overlapping
```

```
    if (L <= low && high <= R) {
```

```
        return tree[index];
```

```
    }
```

```
    // Partially overlapping
```

```
    int mid = (low + high) / 2;
```

```
    long long leftAnswer =
```

```
        query(tree, low, mid, L, R, 2 * index + 1);
```

```
    long long rightAnswer =
```

```
    query(tree, mid + 1, high, L, R, 2 * index + 2);

    return leftAnswer * rightAnswer;
}
```

```
int main() {
```

```
    int N;
```

```
    cin >> N;
```

```
    vector<long long> A(N);
```

```
    for (int i = 0; i < N; i++) {
```

```
        cin >> A[i];
```

```
    }
```

```
    // Segment tree
```

```
    vector<long long> tree(4 * N);
```

```
    // Build
```

```
    build(A, tree, 0, N - 1, 0);
```

```
    int Q;
```

```
    cin >> Q;
```

```
while (Q--) {  
  
    int L, R;  
  
    cin >> L >> R;  
  
    cout << query(tree, 0, N - 1, L, R, 0)  
        << endl;  
}  
  
return 0;  
}
```

Test cases where intermediate nodes can be void and we need more than $2N-1$ segment array size.

A = [10,20,30,40,50,60]

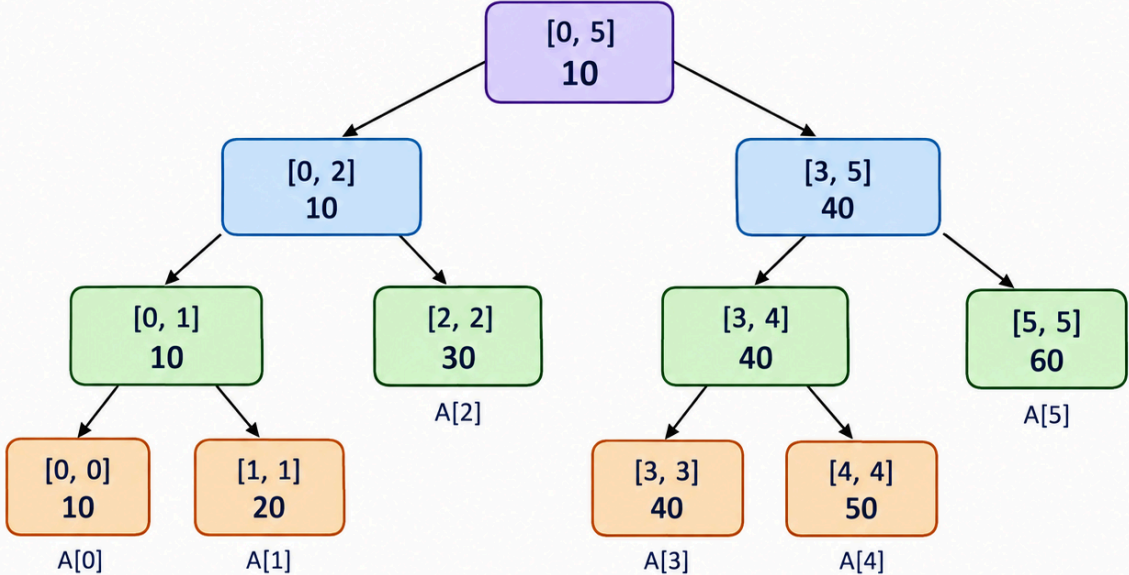
N=6

B = [10,20,30,40,50,60,70,80,90,100]

N = 10

Array A (N = 6)					
0	1	2	3	4	5
10	20	30	40	50	60

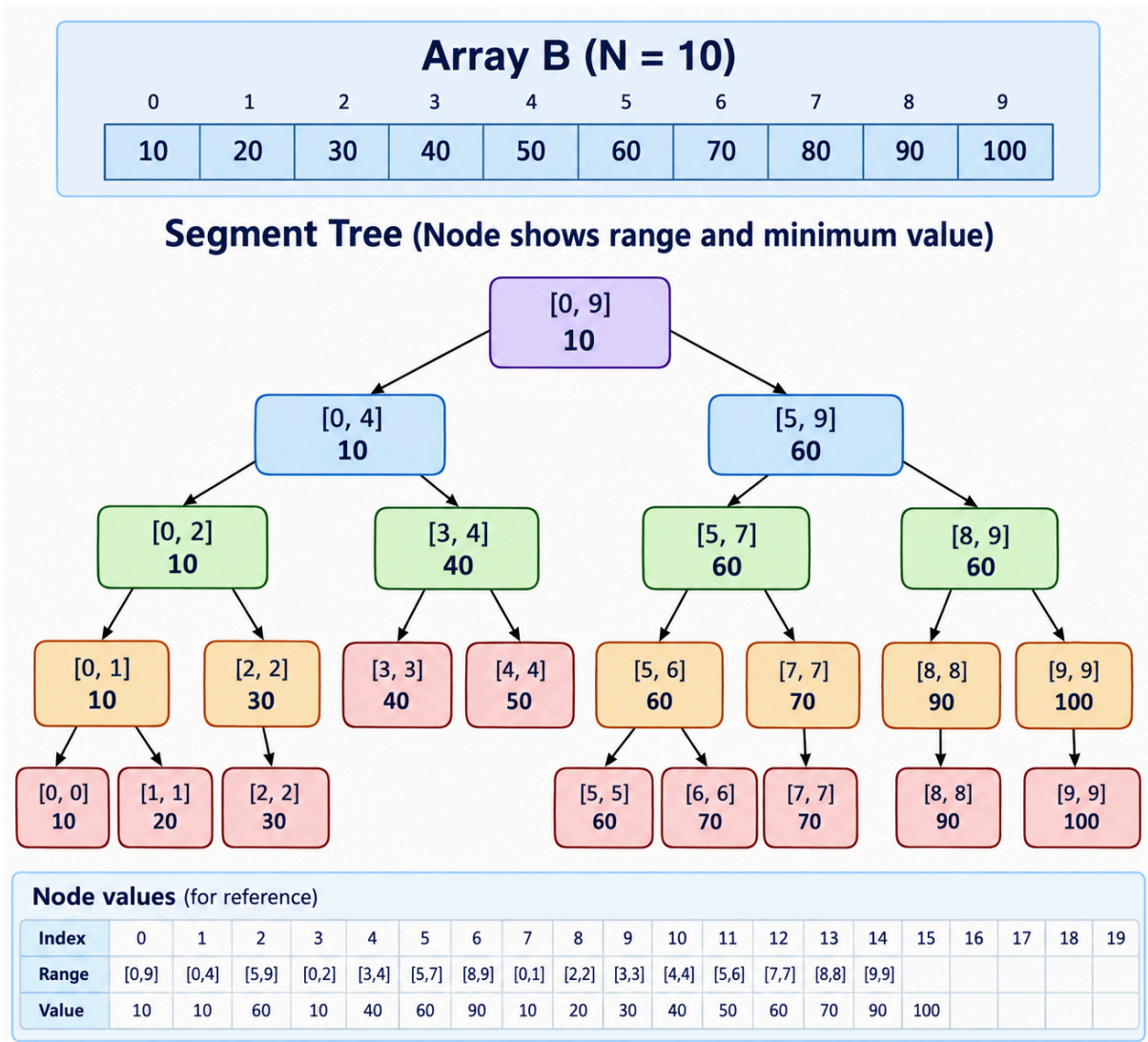
Segment Tree (Node shows range and minimum value)



Node values (for reference)												
Range	[0,5]	[0,2]	[3,5]	[0,1]	[2,2]	[3,4]	[5,5]	[0,0]	[1,1]	[3,3]	[4,4]	[5,5]
Value	10	10	40	10	30	40	60	10	20	40	50	60

B = [10,20,30,40,50,60,70,80,90,100]

N = 10



Question-5:

You are given a string **S** consisting of lowercase English letters.

Your task is to build a **Segment Tree** where each node stores the **frequency of all 26 lowercase English characters** in its corresponding substring.

For every segment of the string, maintain an array of size **26**:

- **freq[0]** → frequency of 'a'
- **freq[1]** → frequency of 'b'
- ...
- **freq[25]** → frequency of 'z'

Example

Given:

$S = \text{"abacca"}$

The frequency of the complete string is:

$a \rightarrow 3$
 $b \rightarrow 1$
 $c \rightarrow 2$
 $d \rightarrow 0$
...
 $z \rightarrow 0$

For a leaf node representing the character 'c', its frequency array will contain:

$a \rightarrow 0$
 $b \rightarrow 0$
 $c \rightarrow 1$
 $d \rightarrow 0$
...
 $z \rightarrow 0$

Build the Segment Tree recursively.

For each internal node:

1. Build the left child.
2. Build the right child.
3. Merge the two children by adding their corresponding character frequencies.

For every character x :

$\text{parent.freq}[x] = \text{left.freq}[x] + \text{right.freq}[x]$

Input

A string S containing lowercase English letters.

Output

Construct the Segment Tree such that every node contains the frequency of all 26 characters in its corresponding segment.

Constraints

$1 \leq |S| \leq 10^5$


```

//main logic
int mid=(low+high)/2;
build(s,low,mid,2*index+1,segment_array);
build(s,mid+1,high,2*index+2,segment_array);
for(int t=0;t<26;t++){

segment_array[index][t]+=(segment_array[2*index+1][t]+segment_array[2*index+2][t]);

}
}
int main(){
    string s="abacca";
    int n=s.size();
    vector<vector<int>>segment_array(4*n,vector<int>(26,0));
    build(s,0,s.size()-1,0,segment_array);
    return 0;
}

```

Question-6:

You are given a string **S** of length **N** consisting only of the characters '(' and ' '.

The characters are indexed from 0 to N-1.

You are also given **Q** queries. Each query contains two integers **L** and **R**, representing the substring:

S[L...R]

For each query, find the **length of the longest valid parentheses subsequence** that can be formed using characters from **S[L...R]**.

A subsequence does not need to be contiguous, but the relative order of the selected characters must remain unchanged.

A valid parentheses sequence is a sequence in which every opening bracket '(' is correctly matched with a closing bracket ') '.

Since there can be a large number of queries, you must construct a **Segment Tree** to answer the queries efficiently.

For every segment-tree node, maintain the following information:

- **open** — number of unmatched opening brackets '('
- **close** — number of unmatched closing brackets ') '
- **match** — number of matched pairs of brackets ()

When merging two child nodes:

$$\text{newMatches} = \min(\text{left.open}, \text{right.close})$$
$$\text{parent.open} = \text{left.open} + \text{right.open} - \text{newMatches}$$
$$\text{parent.close} = \text{left.close} + \text{right.close} - \text{newMatches}$$
$$\text{parent.match} = \text{left.match} + \text{right.match} + \text{newMatches}$$

For a query, the length of the longest valid parentheses subsequence is:

$$2 \times \text{match}$$

Input Format

N

S

Q

L1 R1

L2 R2

...

LQ RQ

Where:

- N — length of the string
- S — string containing only '(' and ')'
- Q — number of queries
- $L_i R_i$ — range of the i -th query

Output Format

For each query, print the length of the longest valid parentheses subsequence in $S[L_i \dots R_i]$.

Example

Input

9

((()))()

Input

10

`((()))(( ))`

Explanation

For the first query:

$L = 0, R = 8$

$S[0...8] = "((()))(())"$

The longest valid parentheses subsequence has 4 matched pairs:

`((()))`

Therefore, its length is:

$4 \times 2 = 8$

For the query `[0, 2]`:

$S[0...2] = "(())"$

One valid pair can be formed:

`()`

Therefore, the answer is `2`.

Constraints

$1 \leq N \leq 10^5$

$1 \leq Q \leq 10^5$

$S[i] \in \{'(', '\text{'}\}$

$0 \leq L \leq R < N$

Solution:

```
#include <bits/stdc++.h>
using namespace std;
```

```

int segment_arr4N][3];
string s;

// 0 -> open
// 1 -> close
// 2 -> pairs

void build(int node, int low, int high) {

    if (low ==high) {

        if (s[low] == '(') {
            segment_arr[i][0] = 1; // open
        }
        else {
            segment_arr[i][1] = 1; // close
        }

        return;
    }

    int mid = (start + end) / 2;

    build(2 *i+1, start, mid);
    build(2 * i+ 2, mid + 1, end);

    // Left child
    int leftOpen  = segment_arr[2 *i+1][0];
    int leftClose = segment_arr[2 *i+1][1];
    int leftPairs = segment_arr[2 *i+1][2];

    // Right child
    int rightOpen  = segment_arr[2 * i+ 2][0];
    int rightClose = segment_arr[2 * i+ 2][1];
    int rightPairs = segment_arr[2 * i+ 2][2];

    int pairs = min(leftOpen, rightClose);

```

```

Code-1 segment_arr[i][0] = leftOpen + rightOpen -pairs;
segment_arr[i][1] = leftClose + rightClose -pairs;
segment_arr[i][2] = leftPairs + rightPairs +pairs;

```

OR

```
Code-2 for(int t = 0; t <= 1; t++) {  
    segment_arr[i][t] =  
        segment_arr[2*i+1][t] +  
        segment_arr[2*i+2][t] -  
        pairs;  
}
```

```
segment_arr[i][2] =  
    segment_arr[2*i+1][2] +  
    segment_arr[2*i+2][2] +  
    pairs;  
}
```

Code-1 & Code-2 both code snippets are correct.